Distributed Neural Network Training over TCP Sockets

Category 2 — Client–Server Application Using Sockets

Group Name: 55056
Samario Liu utoid:liuxi622
Norris Wang utoid:wang5798

1 Project Overview

This project implements a **distributed parameter server** for training a small neural network across multiple worker processes connected over TCP sockets.

The system consists of two roles: a single **parameter server** and multiple **worker clients**. The server stores the global model weights and coordinates training rounds. Each worker stores a partition (shard) of the training data, computes gradient updates locally using forward and backward passes, and sends its averaged gradients to the server for aggregation. The server then averages the gradients from all workers, updates the global weights, and broadcasts the new weights to begin the next round.

The neural network is a fully connected two-layer network with ReLU activation on the hidden layer and softmax output, trained on the MNIST handwritten digit dataset (28×28 grayscale images, 10 classes). The system is configured for a two-layer MNIST classifier. In our experiments, we use 784 input neurons, 64 hidden neurons, and 10 output neurons, for a total of 50,890 learnable floating-point parameters.

Key features

- TCP socket communication using a custom binary protocol with length-prefixed messages in network byte order.
- Non-blocking I/O with `select()` on the server, allowing concurrent handling of multiple workers without blocking on any single client.
- Synchronous distributed training: the server waits for all workers to submit gradients before averaging and updating weights.
- Model checkpointing: workers save the final trained model to disk using a custom binary format via `net_save()` (in `neural_network.c`, line 289).
- Clean separation between the compute core (`neural_network.c`) and the communication layer (`worker.c`, `server.c`)

User interaction

The training data is preprocessed from MNIST using a Python script (`preprocess_data.py`) that normalizes pixel values to $[0, 1]$, flattens images into 784-float vectors, and splits them into binary shard files. The server is started with command-line arguments specifying the port, number

of expected workers, model dimension, learning rate, and maximum number of rounds. Workers are started with the server address, port, hidden-layer size, and path to their shard file. Training progress (loss and accuracy per round) is printed to standard output by each worker. Upon completion, each worker saves the final model to a `.bin` file.

A general-purpose framework

- **Larger and deeper models.** The server will not be limited by weight or gradient payloads — it uses flat `float[dim]` buffer — so any network architecture can be substituted into `neural_network.c` by updating the `dim` argument at startup. The protocol and socket code require no changes.
- **GPU acceleration per worker.** The matrix multiplications in `net_forward()` and elementwise operations in `net_backward()` are direct candidates for CUDA kernels or cuBLAS. Each worker would keep weights and gradients in device memory throughout the round, copying only the final gradient to host memory via `cudaMemcpy` before the `send_message()` call.
- **Asynchronous training.** The server could apply each gradient as it arrives rather than waiting for all workers, reducing idle time caused by stragglers. The change would be confined to `aggregate_and_update()` in `server.c`; the `select()` loop and per-client buffers already handle gradients arriving independently.
- **Fault-tolerant checkpointing.** Periodically saving the server's global weights and round number to disk using the existing `net_save()` would allow training to resume after a crash without restarting from scratch.

2 Build Instructions

The project compiles with a single `make` command. All source files are written in C and compiled with -std=c11, using only standard POSIX libraries available on `teach.cs`.

Compilation

```
make
```

This produces two executables: `server/server` and `worker/worker`. The build produces zero warnings with `-Wall -Wextra`.

Generating training data

The repository includes pre-generated shard files in `mnist_shards/`, so the C project can be built and run directly on `teach.cs`. We also provide optional Python preprocessing scripts used during development to generate these shard files from MNIST.

```
pip install tensorflow numpy
python3 data/preprocess_data.py
```

This downloads MNIST and creates five training shards and one test shard in `mnist_shards/`.

Running the system

Start the server first, then launch workers in separate terminals. The default port is 4242.

```
# Terminal 1 -- server
# Arguments: <port> <num_workers> <dim> <learning_rate> <max_rounds>
./server/server 4242 3 50890 0.01 50
# or you simply run "make run_server"

# Terminal 2
./worker/worker 127.0.0.1 4242 64 mnist_shards/train_shard_0.bin
# or you can run "make run_worker0"

# Terminal 3
./worker/worker 127.0.0.1 4242 64 mnist_shards/train_shard_1.bin
# or you can run "make run_worker1"

# Terminal 4
./worker/worker 127.0.0.1 4242 64 mnist_shards/train_shard_2.bin
# or you can run "make run_worker2"

# Terminal 5
./worker/worker 127.0.0.1 4242 64 mnist_shards/train_shard_2.bin
# or you can run "make run_worker3"

# Terminal 6
./worker/worker 127.0.0.1 4242 64 mnist_shards/train_shard_2.bin
# or you can run "make run_worker4"
```

No additional input files beyond the shard binaries are required.

Test the model

```
# run
python3 data/preprocess_single_image.py data/5.png demo/5.bin
//now there will be 5.bin in your demo folder
# run
make predict_app
./predict_app trained_model_worker_0.bin demo/5.bin
```

3 Architecture Diagram

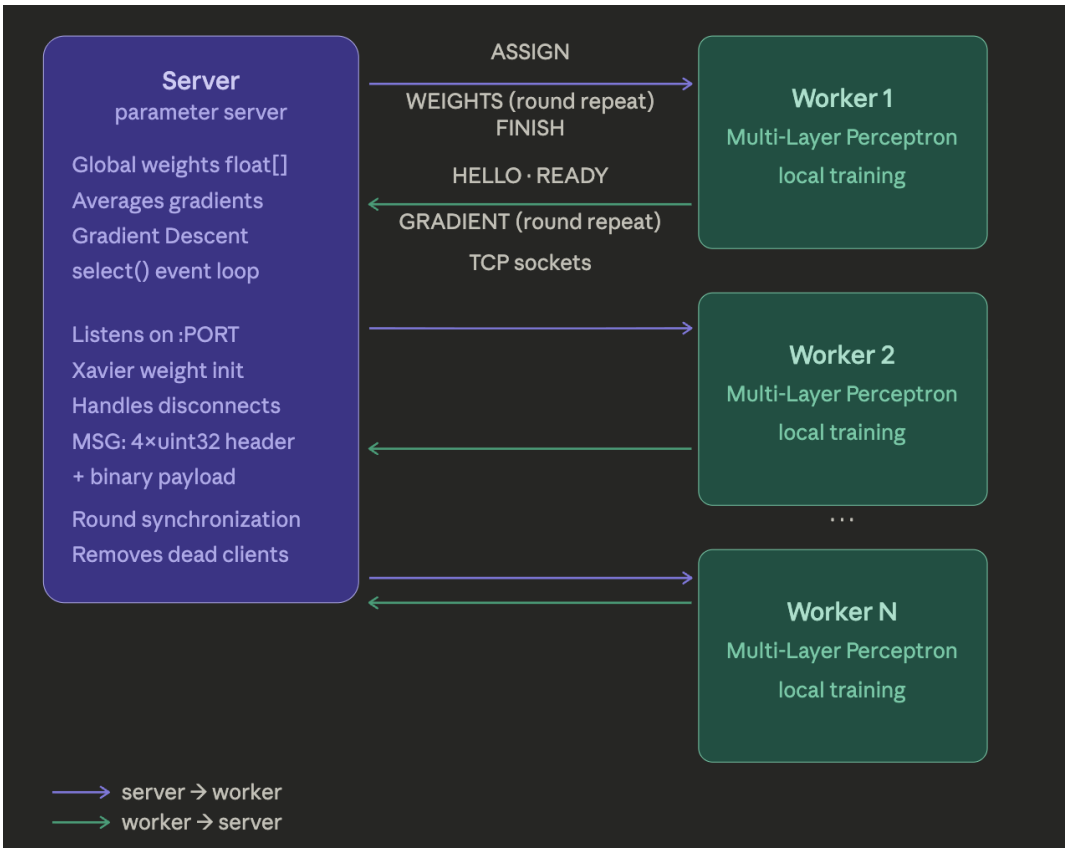

Figure 1: System architecture showing the server, workers, and TCP connections.

4 Communication Protocol

Every message consists of a 16-byte header followed by an optional payload. All integer fields in the header are transmitted in **network byte order** (big-endian) using `htonl()`/`ntohl()`. Float arrays in payloads are sent as raw IEEE-754 bytes, which is valid because all processes run on the same architecture (`teach.cs`).

Header format

Field	Type	Description
`type`	`uint32_t`	Message type identifier (1–7)
`payload_len`	`uint32_t`	Number of bytes in the payload (0 if none)
`round`	`uint32_t`	Training round number (0 during handshake)
`worker_id`	`uint32_t`	Worker identifier assigned by the server

The header struct is defined in both `server.c` (line 52) and `worker.h` (line 36).

Message sequence

The following diagram shows the full message exchange between one worker and the server during a complete training session:

```
Worker i                <------>                Server
--------                                        ------
connect()  ------------------------------------> accept()

MSG_HELLO
(type=1, payload=empty) ------------------------> validates first contact
                                                   assigns worker_id, dim,
                                                   max_rounds, learning_rate
           <------------------------------------- MSG_ASSIGN
                                                   payload = assign_payload_t
                                                   { worker_id, dim,
                                                     max_rounds, lr }

//worker allocates net/grads/cache
//checks dimension consistency

MSG_READY
(type=3, payload=empty) ------------------------> marks worker ready

              (server waits for all workers)

           <------------------------------------- MSG_WEIGHTS
                                                   payload = float[dim]

//worker:
//  - unpack weights into local net
//  - forward + backward on local shard
//  - average gradients over n_samples

MSG_GRADIENT
(type=5, payload=float[dim],
 round=current, worker_id=id) ------------------> stores gradient

              (server waits for all gradients)
              averages gradients across workers
              W -= lr * avg_gradient

           <------------------------------------- MSG_WEIGHTS (next round)

              ... rounds repeat ...

           <------------------------------------- MSG_FINISH
                                                   payload = float[dim]
                                                   final trained weights
//worker:
//  - unpack final weights
//  - net_save() to disk
//  - close socket, exit
```

MSG_HELLO (type = 1)

Direction	Worker → Server
Encoding	Header only; `payload_len = 0`.
Semantics	The worker announces its presence. The server responds with MSG_ASSIGN. Sent by `do_handshake()` in `worker.c` (line 240).
Error handling	Duplicate HELLO triggers MSG_ERROR ("duplicate HELLO"). See `handle_hello()` in `server.c` (line 422).

MSG_ASSIGN (type = 2)

Direction	Server → Worker
Encoding	20-byte payload struct `assign_payload_t`: `{uint32_t worker_id, uint32_t dim, uint32_t max_rounds, uint32_t reserved, float learning_rate}`. Integer fields are in network byte order; `learning_rate` is sent as a raw IEEE-754 float.
Semantics	Assigns the worker a unique ID and communicates model dimensions and training hyperparameters. The worker uses `dim` to verify consistency with its local network dimensions. See `handle_hello()` in `server.c` (line 422) and `do_handshake()` in `worker.c` (line 240).
Error handling	If the server cannot allocate resources, it sends MSG_ERROR and closes the connection.

MSG_READY (type = 3)

Direction	Worker → Server
Encoding	Header only; `payload_len = 0`.
Semantics	The worker confirms that it has loaded its data shard and allocated all neural network structures. The server begins training once all expected workers have sent READY. See `maybe_start_or_advance_training()` in `server.c` (line 378).
Error handling	READY before HELLO or duplicate READY triggers MSG_ERROR. See `handle_ready()` in `server.c` (line 441).

MSG_WEIGHTS (type = 4)

Direction	Server → Worker
Encoding	`float[dim]` — the entire model packed as W1, b1, W2, b2 in contiguous row-major order. `payload_len = dim * sizeof(float)`. The packing order matches `net_pack_params()` in `neural_network.c` (line 241).
Semantics	Delivers the current global weights for the worker to use in the next training round. The worker unpacks them via `net_unpack_params()` and overwrites its local network parameters. See `broadcast_weights()` in `server.c` (line 297) and `train_one_round()` in `worker.c` (line 329).
Error handling	If `queue_message()` fails because of output buffer overflow, the worker is marked inactive. On the worker side, `payload_len` is validated against the expected size before unpacking.

MSG_GRADIENT (type = 5)

Direction	Worker → Server
Encoding	`float[dim]` — averaged gradients packed in the same order as the weights (dW1, db1, dW2, db2). Each gradient value is divided by the worker's local sample count before sending. The packing uses `grads_pack()` in `neural_network.c` (line 265).
Semantics	The worker submits its computed gradient update for the current round. The server collects gradients from all workers, averages them in `aggregate_and_update()` (`server.c`, line 338), applies the update `W -= lr * avg_grad`, and broadcasts the new weights.
Error handling	The server validates five conditions before accepting a gradient: (1) training must have started, (2) the worker must have sent READY, (3) the round number must match, (4) the payload size must equal `dim * sizeof(float)`, and (5) there must not already be a gradient from that worker for the same round. Any violation sends MSG_ERROR. See `handle_gradient()` in `server.c` (line 454).

MSG_FINISH (type = 6)

Direction	Server → Worker
Encoding	`float[dim]` — the final trained weights.
Semantics	Indicates that training is complete. Workers unpack the final model, save it to disk via `net_save()` (`neural_network.c`, line 289), and exit gracefully. See `broadcast_finish()` in `server.c` (line 320) and the MSG_FINISH case in `training_loop()` in `worker.c` (line 393).
Error handling	Workers close the socket and free all resources. If `net_save()` fails, an error is logged to standard error, but the worker still exits cleanly.

MSG_ERROR (type = 7)

Direction	Either direction
Encoding	UTF-8/ASCII null-terminated string describing the error.
Semantics	Indicates a protocol violation or internal error. The recipient logs the error string to standard error.
Error handling	The recipient terminates the connection after logging the message. See `queue_error()` in `server.c` (line 219).

5 Concurrency Model

The server achieves concurrency using a single-threaded event loop driven by `select()`. No threads or `fork()`-per-client mechanisms are used.

The `select()` loop

On each iteration of the main loop (`server.c`, `main()`, line 712), the server constructs two `fd_sets`:

- `readfds` contains the listening socket and every active client socket.

- `writefds` contains only those client sockets that have pending output data in their `outbuf` (that is, `out_used > out_sent`).

The server calls `select(maxfd + 1, &readfds, &writefds, NULL, NULL)`, which blocks until at least one file descriptor is ready for I/O.

When `select()` returns, the server processes events in the following order:

1. If the listening socket is in `readfds`, call `accept_new_clients()` (`server.c`, line 584) to accept pending connections.
2. For each active client, if its file descriptor is in `readfds`, call `handle_client_readable()` (`server.c`, line 556) to read incoming data and process complete messages.
3. For each active client, if its file descriptor is in `writefds`, call `flush_client_output()` (`server.c`, line 223) to send pending responses.
4. Call `remove_dead_clients()` to clean up any disconnected clients.
5. Call `maybe_start_or_advance_training()` to check whether training should begin or advance to the next round.

This design ensures that no single client can block the server while others are waiting. A slow or unresponsive client simply does not have its file descriptor marked ready by `select()`, and the server continues servicing other clients.

Non-blocking I/O

All client sockets are set to non-blocking mode via `fcntl(fd, F_SETFL, flags | O_NONBLOCK)` immediately after `accept()` (see `set_nonblocking()` in `server.c`, line 107). This means that `recv()` and `send()` return `EAGAIN`/`EWOULDBLOCK` instead of blocking when no data is available or when the send buffer is full.

Buffered message handling

Each client has a 1 MB input buffer (`inbuf`) and a 1 MB output buffer (`outbuf`). Incoming bytes are appended to `inbuf`; the function `process_client_buffer()` (`server.c`, line 517) scans for complete messages by checking whether at least `sizeof(msg_header_t) + payload_len` bytes are present. Incomplete messages remain in the buffer until more data arrives. After processing, consumed bytes are removed with `memmove()`. Outgoing messages are queued into `outbuf` via `queue_message()` and flushed when `select()` indicates that the socket is writable.

6 Error Handling and Robustness

Every system call that can fail (`read`, `write`, `send`, `recv`, `accept`, `connect`, `malloc`, `fopen`) has its return value checked. Below we document three specific failure scenarios.

Scenario 1: Worker disconnects mid-training

What happens: A worker process crashes or is killed (for example, via Ctrl+C) while training is in progress. The server's next `recv()` on that worker's socket returns 0 (EOF) or -1.

How it is handled: The function `handle_disconnect()` (`server.c`, line 404) marks the worker as inactive (`c->active = false`). On the next loop iteration, `remove_dead_clients()` (`server.c`, line 394) closes the socket via `close(c->fd)` and frees the gradient buffer via `free(c->gradient)`.

If the remaining workers have all submitted their gradients for the current round, the server proceeds with aggregation using only the available gradients. If no workers remain, the server sets `training_finished = true` and exits cleanly.

Scenario 2: Malformed or out-of-order protocol messages

What happens: A worker sends READY before HELLO, a duplicate HELLO, a GRADIENT for the wrong round number, or a message with an incorrect payload size.

How it is handled: The server's `dispatch_message()` function (`server.c`, line 485) validates every incoming message. Specific checks include: `handle_hello()` rejects duplicate HELLO messages; `handle_ready()` rejects READY before HELLO and duplicate READY messages; and `handle_gradient()` validates that training has started, the worker is ready, the round number matches, the payload size equals `dim * sizeof(float)`, and no duplicate gradient exists for the current round. Each violation triggers `queue_error()`, which sends MSG_ERROR with a descriptive string back to the worker. The worker logs the error to standard error and terminates.

Scenario 3: Partial reads and writes on TCP sockets

What happens: A large message (for example, a 200 KB gradient array) is sent over TCP, but `send()` or `recv()` transfers fewer bytes than requested because the kernel buffer is full or because only a partial TCP segment has arrived.

How it is handled: On the worker side, `send_all()` and `recv_all()` (`worker.c`, lines 16–54) loop until all requested bytes have been transferred, retrying on `EINTR` (signal interruption) and treating `recv()` returning 0 as a server disconnection. On the server side, the non-blocking `recv()` in `handle_client_readable()` (`server.c`, line 556) appends whatever bytes arrive to `inbuf`, and `process_client_buffer()` only extracts a message once the full header plus payload is available. Unconsumed bytes are shifted to the front of the buffer with `memmove()`. For writes, `flush_client_output()` (`server.c`, line 223) sends as many bytes as the kernel accepts and tracks progress via `out_sent`; the remaining bytes are sent on the next `select()` iteration when the socket becomes writable.